HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

PROJECT 01
PHÁT TRIỂN HỆ THỐNG THÔNG MINH

HỆ ĐA AGENT CHO BỆNH TIỂU ĐƯỜNG

Deep Learning, Knowledge Base (Neo4j) và RAG,
Reinforcement Learning kết hợp luật, hội đồng tranh biện

Giảng viên hướng dẫn:	PGS. TS. Trần Đình Quế
Sinh viên thực hiện:	Nguyễn Tự Kiên
Mã sinh viên:	B23DCCN465
Nhóm lớp:	CT01
Nhóm:	07

Hà Nội, tháng 11 năm 2026

Mục lục

DANH MỤC TỪ VIẾT TẮT . . . . . **iv**

CHƯƠNG 1. Giới thiệu **1**
1.1. Bài toán và động lực 1
1.2. Mục tiêu và phạm vi 1
1.3. Đóng góp và cấu trúc báo cáo 2

CHƯƠNG 2. Hệ đa agent **3**
2.1. Agent 3
2.2. Hệ đa agent 4
2.3. Giao tiếp giữa các agent 5

CHƯƠNG 3. Kiến trúc tổng quan **7**
3.1. Sơ đồ của đề bài và hiện thực 7
3.2. Vai trò của các agent 9
3.3. Luồng một lượt hội chẩn 9
3.4. Công nghệ và lưu trữ 10

CHƯƠNG 4. Các thành phần của hệ thống **12**
4.1. Deep Learning ⇒ Agent 12
4.1.1. Dữ liệu và bài toán 12
4.1.2. Mô hình và huấn luyện 12
4.1.3. Dữ liệu quy mô lớn: BRFSS 2011–2023 13
4.1.4. Từ mô hình thành agent 16
4.2. Knowledge Base và RAG 16
4.2.1. Nguồn tri thức và quy trình tạo KB 16
4.2.2. Truy xuất: tìm kiếm lai và GraphRAG 17
4.2.3. Đánh giá truy xuất 18
4.3. Reinforcement Learning và luật 19
4.3.1. Agent gợi ý can thiệp bằng Q-learning 19
4.3.2. Luật ràng buộc chính sách học được 20
4.4. Hội đồng tranh biện: tư vấn và trợ giúp 20
4.5. Tiếp nhận, làm rõ và bộ nhớ 22

CHƯƠNG 5. Code và Demo **23**
5.1. Cấu trúc mã nguồn 23
5.2. Các hàm chính 24
5.3. Cách chạy 25
5.4. Demo: chatbot và Phòng hội chẩn 25
5.5. Ví dụ một lượt hội chẩn 27
5.6. Kiểm thử 29
5.7. Đánh giá đầu–cuối 29
5.8. Hạn chế và hướng phát triển 31

KẾT LUẬN **33**

TÀI LIỆU THAM KHẢO **34**

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
ADA	American Diabetes Association	Hiệp hội Đái tháo đường Hoa Kỳ
AUC	Area Under the (ROC) Curve	Diện tích dưới đường ROC
BM25	Best Matching 25	Hàm xếp hạng BM25
BMI	Body Mass Index	Chỉ số khối cơ thể
DL	Deep Learning	Học sâu
ĐTĐ	–	Đái tháo đường
FIPA-ACL	FIPA Agent Communication Language	Ngôn ngữ giao tiếp tác tử FIPA
FPG	Fasting Plasma Glucose	Glucose huyết tương lúc đói
FTS	Full-Text Search	Tìm kiếm toàn văn
GBDT	Gradient Boosted Decision Trees	Cây quyết định tăng cường gradient
HbA1c	Glycated Hemoglobin	Huyết sắc tố gắn glucose
HNSW	Hierarchical Navigable Small World	Đồ thị thế giới nhỏ phân cấp
KB	Knowledge Base	Cơ sở tri thức
KG	Knowledge Graph	Đồ thị tri thức
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MAS	Multi-Agent System	Hệ đa tác tử (hệ đa agent)
MDP	Markov Decision Process	Quá trình quyết định Markov
ML	Machine Learning	Học máy
MLP	Multilayer Perceptron	Mạng nơ-ron nhiều lớp
NHANES	National Health and Nutrition Examination Survey	Khảo sát Sức khỏe và Dinh dưỡng Quốc gia Hoa Kỳ
OGTT	Oral Glucose Tolerance Test	Nghiệm pháp dung nạp glucose đường uống
RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất
RL	Reinforcement Learning	Học tăng cường
RRF	Reciprocal Rank Fusion	Gộp xếp hạng nghịch đảo

Danh sách hình vẽ

Hình 2.1. Vòng lặp quan sát – hành động của một agent 3

Hình 3.1. Sơ đồ hệ đa agent theo đề bài (vẽ lại từ bảng), kèm thành phần hiện thực của project 7
Hình 3.2. Luồng xử lý một lượt chat: Orchestrator rẽ nhánh theo ý định và mức đủ dữ liệu 10
Hình 3.3. Thành phần triển khai và nơi lưu từng loại dữ liệu 10

Hình 4.1. Đường ROC và Precision–Recall của các mô hình trên tập kiểm tra NHANES 13
Hình 4.2. Quy trình tạo Knowledge Base từ web và sách 17
Hình 4.3. Đồ thị tri thức về ĐTĐ (nút theo nhãn của ontology) 17

Hình 5.1. Giao diện chatbot và Phòng hội chẩn: hội đồng đang chất vấn agent Dự đoán (mũi tên đỏ) 26
Hình 5.2. Ba thời điểm khi phát lại một lượt hội chẩn trong Phòng hội chẩn 27
Hình 5.3. Thời gian xử lý của từng agent và tần suất các loại chất vấn . . . 30
Hình 5.4. Độ nhạy, đặc hiệu của các cách ra quyết định, và tỉ lệ ĐTĐ thật theo mức nguy cơ của hội đồng 31

Danh sách bảng

Bảng 2.1. Các loại agent và hiện thực trong project 4
Bảng 2.2. Một agent làm tất cả so với hệ đa agent của project 5
Bảng 2.3. Năm performative dùng trong hệ thống 6

Bảng 3.1. Ánh xạ khối trên sơ đồ sang thành phần của hệ thống 8
Bảng 3.2. Chín agent, thông điệp chúng nhận và kỹ thuật bên trong 9
Bảng 3.3. Công nghệ sử dụng 11

Bảng 4.1. Kết quả trên tập kiểm tra NHANES (4.130 người) 13
Bảng 4.2. Dữ liệu BRFSS theo năm sau khi chuẩn hóa 14
Bảng 4.3. Chia dữ liệu huấn luyện – kiểm định – kiểm tra (70/10/20, phân tầng theo nhãn, `random_state=42`) 15
Bảng 4.4. Kết quả trên tập kiểm tra BRFSS (634.564 người) 15
Bảng 4.5. Truy xuất trên 60 câu hỏi tiếng Việt (36 đoạn văn) 19
Bảng 4.6. Safety Guardrail: luật an toàn luôn được ưu tiên 20
Bảng 4.7. Giao thức tranh biện của hội đồng 21

Bảng 5.1. Các hàm và lớp chính 24
Bảng 5.2. Luồng 22 thông điệp của lượt hội chẩn demo (ms: thời gian xử lý ở thông điệp trả lời) 28
Bảng 5.3. Bộ kiểm thử tự động (43 kiểm thử, đều đạt) 29
Bảng 5.4. Đủ xét nghiệm (FPG + HbA1c): tính đúng đắn và vận hành của hội đồng (399 ca) 30
Bảng 5.5. Chỉ có FPG: giá trị của việc phối hợp nhiều agent (399 ca, 91 người ĐTĐ) 31

CHƯƠNG 1. GIỚI THIỆU

1.1. Bài toán và động lực

Đái tháo đường (ĐTĐ, bệnh tiểu đường) là tình trạng glucose trong máu tăng cao kéo dài do cơ thể thiếu insulin hoặc kháng insulin. Phần lớn ca bệnh là ĐTĐ típ 2, loại tiến triển âm thầm nhiều năm và có thể làm chậm hoặc phòng ngừa nếu phát hiện sớm từ giai đoạn tiền ĐTĐ [1], [2]. Ở Việt Nam, Hướng dẫn chẩn đoán và điều trị ĐTĐ típ 2 của Bộ Y tế [3] khuyến cáo sàng lọc chủ động cho người có yếu tố nguy cơ, nhưng người dân thường chỉ có trong tay vài con số rời rạc (một lần thử đường huyết, cân nặng, huyết áp) và không biết chúng nói lên điều gì.

Một công cụ hỗ trợ sàng lọc tốt phải làm được bốn việc cùng lúc: *dự đoán* nguy cơ từ dữ liệu sức khỏe; *giải thích* dựa trên tri thức y khoa có nguồn; *gợi ý* bước can thiệp cụ thể; và *cảnh báo* ngay khi có dấu hiệu nguy hiểm. Mỗi việc cần một kỹ thuật khác nhau: học sâu cho dự đoán, cơ sở tri thức và RAG cho giải thích, học tăng cường cho ra quyết định tuần tự, và luật cố định cho an toàn. Gộp tất cả vào một chương trình khiến hệ thống khó kiểm tra và khó tin cậy. Vì vậy project xây dựng một **hệ đa agent**: mỗi agent giữ một chuyên môn, các agent trao đổi bằng thông điệp có cấu trúc, và một hội đồng kiểm tra chéo kết luận trước khi tư vấn cho người dùng.

1.2. Mục tiêu và phạm vi

Project hiện thực hóa sơ đồ mà giảng viên đưa ra trên lớp (Hình 3.1, Chương 3) thành một hệ thống chạy được đầu–cuối:

- **DB → Deep Learning → dự đoán, phân loại**: huấn luyện mô hình học máy và mạng nơ-ron trên hai bộ dữ liệu có sẵn của CDC, tổng cộng **3.193.465 người**: NHANES 2011–2018 (20.646 người, nhãn xác nhận bằng xét nghiệm) làm mô hình chính, và BRFSS 2011–2023 (**3.172.819 người**, bảy năm khảo sát) làm mô hình quy mô lớn; ước lượng xác suất ĐTĐ rồi phân tầng nguy cơ thành bốn mức.
- **Web, sách → Neo4j KB → RAG**: nạp hướng dẫn ADA 2024 và QĐ 5481/QĐ-BYT vào cơ sở tri thức; đồ thị tri thức lưu trên **Neo4j**, đoạn văn được tìm kiếm lai trên **Qdrant** (vector) và **SQLite FTS5** (toàn văn).
- **Reinforcement learning + rule → Agent**: agent gợi ý can thiệp học bằng Q-learning nhưng bị ràng buộc bởi luật an toàn và kết luận của hội đồng.

- **Tư vấn, trợ giúp**: hội đồng tranh biện liệt kê bằng chứng đồng thuận (+) và mâu thuẫn (−), rồi agent phản hồi viết câu trả lời có trích dẫn.
- **Code + Demo**: chatbot web kèm *Phòng hội chẩn* trực quan hóa từng thông điệp giữa các agent; mã nguồn có kiểm thử tự động.

Phạm vi. Đây là sản phẩm học tập. Hệ thống phục vụ *sàng lọc* chứ không *chẩn đoán*, không đưa liều thuốc cụ thể, và mọi câu trả lời đều nhắc người dùng đi khám bác sĩ. Mọi con số trong báo cáo được đo lại từ mã nguồn của project.

1.3. Đóng góp và cấu trúc báo cáo

Các đóng góp chính:

- Một hệ **chín agent** giao tiếp qua `MessageBus` theo tinh thần FIPA-ACL, có audit trail đầy đủ và cô lập lỗi.
- Khối DB gồm **hơn 3,19 triệu hồ sơ** thật của CDC, gấp hơn 12 lần bản BRFSS 2015 đã làm sạch (253.680 dòng) thường được dùng cho bài toán này. Mô hình huấn luyện trên dữ liệu quy mô lớn đóng vai *ý kiến thứ hai*, và hội đồng chất vấn khi nó ngược với mô hình chính.
- Hai nhánh song song đúng như sơ đồ: *DB* $\rightarrow$ *ML/DL* và *KB (Neo4j)* $\rightarrow$ *RAG*, gặp nhau ở một hội đồng có **tranh biện nhiều vòng**: agent phản biện chất vấn agent chuyên môn khi chúng mâu thuẫn.
- Agent **RL + luật**: chính sách Q-learning chỉ được hành động trong khuôn khổ do tri thức tường minh cho phép.
- **Phòng hội chẩn**: giao diện phát lại luồng thông điệp, cho người xem thấy agent nào đang nói gì với agent nào.
- Đánh giá trên 634.564 hồ sơ BRFSS kiểm tra và đánh giá đầu–cuối trên 400 ca NHANES chưa từng thấy; đo lại truy xuất RAG trên bộ lưu trữ mới.

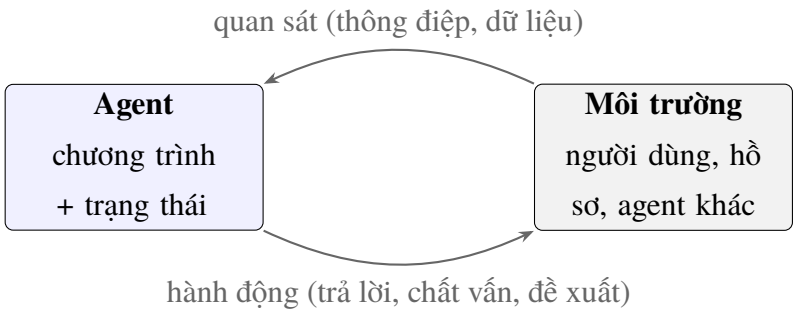
Báo cáo đi theo đúng dàn ý trên bảng. Chương 2 trình bày cơ sở về agent và hệ đa agent. Chương 3 là kiến trúc tổng quan. Chương 4 mô tả từng thành phần: Deep Learning, KB và RAG, RL kết hợp luật, hội đồng và tầng tiếp nhận. Chương 5 trình bày mã nguồn, demo, kiểm thử và đánh giá. Cuối cùng là kết luận.

CHƯƠNG 2. HỆ ĐA AGENT

2.1. Agent

Theo Wooldridge [4], *agent* (tác tử) là một hệ thống tính toán đặt trong một môi trường, có khả năng *hành động tự chủ* trong môi trường đó để đạt mục tiêu được giao. Điều cốt lõi là vòng lặp *quan sát* → *quyết định* → *hành động* → *nhận phản hồi*, chứ không chỉ là một hàm trả về kết quả. Một agent thông minh có bốn tính chất [5]:

- **Tự chủ**: tự quyết định hành động mà không cần con người can thiệp từng bước.
- **Phản ứng**: nhận biết thay đổi của môi trường và phản hồi kịp thời.
- **Chủ động**: hướng tới mục tiêu, tự khởi xướng hành động.
- **Xã hội**: tương tác với agent khác và với con người qua một ngôn ngữ giao tiếp.



Hình 2.1. Vòng lặp quan sát – hành động của một agent

Russell và Norvig [6] phân loại agent theo độ phức tạp của chương trình bên trong. Bảng 2.1 đặt các agent của project vào cách phân loại đó: mỗi loại đều có mặt trong hệ thống.

Bảng 2.1. Các loại agent và hiện thực trong project

Loại agent	Cách ra quyết định	Trong project
Phản xạ theo luật	Nếu điều kiện thì hành động; không học	Safety Guardrail trong `ConsensusCriticAgent`, `ClarificationAgent` (luật phạm vi)
Phản xạ có mô hình	Duy trì trạng thái nội tại	`ConversationAgent`, `DocumentAgent` cập nhật hồ sơ `PatientProfile`
Dựa trên mục tiêu	Lập kế hoạch để đạt mục tiêu	`Orchestrator` lập kế hoạch động cho từng lượt chat
Dựa trên tri thức	Suy luận trên tri thức tường minh	`KnowledgeAgent`: ngưỡng ADA, đồ thị Neo4j, RAG
Agent học	Cải thiện từ dữ liệu, phần thưởng	`PredictionAgent` (GBDT + MLP), `TreatmentRLAgent` (Q-learning)

2.2. Hệ đa agent

Hệ đa agent (Multi-Agent System, MAS) gồm nhiều agent tương tác để giải quyết bài toán vượt quá khả năng hoặc hiểu biết của từng agent. Nếu chỉ có một agent làm tất cả, nó phải trộn dữ liệu số, văn bản y khoa, chính sách can thiệp và luật an toàn trong cùng một chỗ: khó kiểm thử, khó thay thế, và khó biết kết luận đến từ đâu. Bảng 2.2 so sánh hai cách làm.

Bảng 2.2. Một agent làm tất cả so với hệ đa agent của project

Tiêu chí	Một agent	Hệ đa agent (project)
Độ phức tạp mỗi phần	Cao, trộn nhiều kỹ thuật	Thấp, mỗi agent một kỹ thuật
Kiểm thử	Khó tách lỗi	Kiểm thử từng agent và cả hội đồng (43 test tự động)
An toàn	Luật nằm lẫn trong mô hình	Guardrail độc lập, luôn được ưu tiên
Kiểm tra chéo	Không có	Hội đồng chất vấn khi các agent mâu thuẫn
Chịu lỗi	Một lỗi làm hỏng cả lượt	Lỗi của một agent thành thông điệp `FAILURE`
Chi phí phối hợp	Không có	Cần điều phối và giao thức thông điệp

Mô hình tổ chức. Các kiểu tổ chức MAS phổ biến gồm: *Supervisor* (một agent điều phối giao việc và tổng hợp kết quả); *Blackboard* [7] (các agent đọc ghi một vùng nhớ dùng chung); *Contract Net* [8] (giao việc qua đấu thầu); và *tranh biện ngang hàng*, nơi các agent trao đổi lập luận tới khi đồng thuận [9]. Project kết hợp ba kiểu: `Orchestrator` là supervisor; hồ sơ bệnh nhân `PatientProfile` là blackboard tích lũy qua nhiều lượt chat; và `ConsensusCriticAgent` điều khiển tranh biện giữa các agent chuyên môn.

2.3. Giao tiếp giữa các agent

FIPA-ACL [10] chuẩn hóa thông điệp giữa các agent: mỗi thông điệp có người gửi, người nhận, nội dung, và một *performative* thể hiện ý định giao tiếp. Performative tách "nói gì" khỏi "muốn gì", nên agent nhận biết cần thực hiện, trả lời hay chỉ ghi nhận. Project dùng năm performative:

Bảng 2.3. Năm performative dùng trong hệ thống

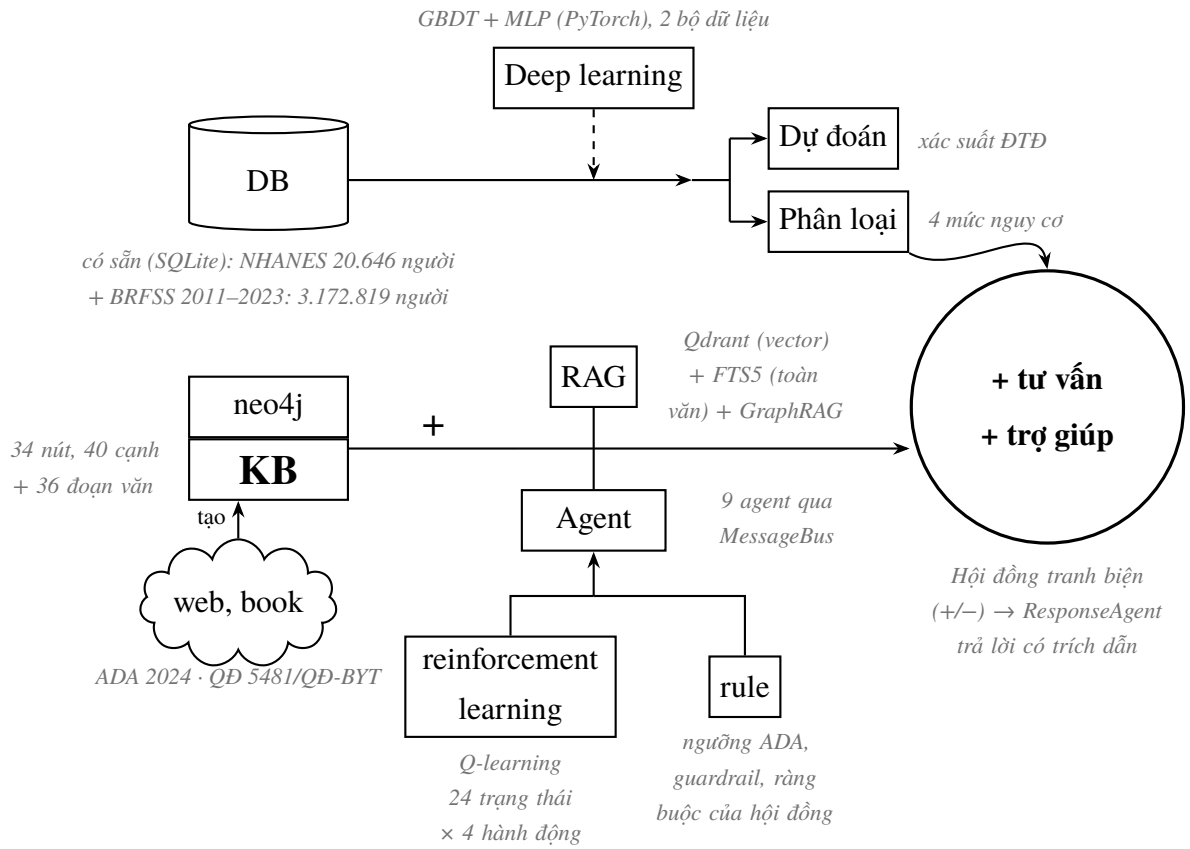
Performative	Ý nghĩa	Ví dụ
`REQUEST`	Yêu cầu agent khác thực hiện một việc	Orchestrator → Prediction: `PREDICT_RISK`
`INFORM`	Trả kết quả của việc được giao	Prediction → Orchestrator: `RISK_PREDICTION`
`CHALLENGE`	Chất vấn khi phát hiện mâu thuẫn	Critic → Prediction: `CHALLENGE_ML_VS_LABS`
`RESPONSE`	Trả lời chất vấn: bảo vệ, nhượng bộ hoặc giải trình	Prediction → Critic: giữ quan điểm "nguy cơ dài hạn"
`FAILURE`	Không thực hiện được (lỗi, không hiểu yêu cầu)	`NOT_UNDERSTOOD`, `PREDICT_RISK_FAILED`

Từ 2023, mô hình ngôn ngữ lớn (LLM) thường được dùng làm "bộ não" của agent, như ReAct [11], AutoGen [12] hay MetaGPT [13]. Các khảo sát [14], [15] chỉ ra ba thách thức: ảo giác lan truyền giữa các agent, chi phí của nhiều lần gọi LLM, và khó kiểm soát khi agent tự quyết định luồng. Project chọn hướng *lai*: LLM chỉ hiểu và diễn đạt ngôn ngữ (trích chỉ số từ tin nhắn, đọc ảnh, viết câu trả lời), còn mọi quyết định lâm sàng do mô hình đã đánh giá và luật tường minh đảm nhận. Nhờ vậy hệ thống chạy được hoàn toàn *không cần LLM*, kết quả tái lập được.

CHƯƠNG 3. KIẾN TRÚC TỔNG QUAN

3.1. Sơ đồ của đề bài và hiện thực

Hình 3.1 vẽ lại sơ đồ trên bảng, giữ nguyên bố cục, và ghi kèm (chữ nghiêng màu xám) thành phần hiện thực tương ứng trong project. Có ba dòng chảy hội tụ vào agent trung tâm: (1) dữ liệu *có sẵn* đi qua Deep Learning để *dự đoán* và *phân loại*; (2) tri thức từ web, sách được *tạo* thành Knowledge Base trên Neo4j và truy xuất bằng RAG; (3) reinforcement learning và luật (rule) cùng điều khiển agent ra quyết định. Kết quả hội tụ ở vòng tròn lớn: hội đồng *tư vấn* và *trợ giúp* người dùng.



Hình 3.1. Sơ đồ hệ đa agent theo đề bài (vẽ lại từ bảng), kèm thành phần hiện thực của project

Bảng 3.1 ánh xạ từng khối trên sơ đồ sang agent và công nghệ cụ thể.

Bảng 3.1. Ánh xạ khối trên sơ đồ sang thành phần của hệ thống

Khối trên sơ đồ	Agent	Hiện thực
DB (có sẵn) → Deep learning → Dự đoán, Phân loại	`PredictionAgent`	GBDT (scikit-learn) + MLP (PyTorch) huấn luyện trên NHANES (mô hình chính) và BRFSS 3,17 triệu người (ý kiến thứ hai), cùng lưu trong SQLite; phân tầng 4 mức (Mục 4.1)
web, book → tạo → neo4j KB	`scripts/ingest_kb.py`	Đọc Markdown, PDF, HTML, URL; chia đoạn theo mục; bộ ba tri thức MERGE vào Neo4j; vector vào Qdrant (Mục 4.2)
KB + RAG → Agent	`KnowledgeAgent`	Đối chiếu ngưỡng ADA; tìm kiếm lai Qdrant + FTS5 gộp bằng RRF; GraphRAG duyệt đồ thị Neo4j
reinforcement learning + rule → Agent	`TreatmentRLAgent`	Q-learning chọn bước can thiệp, bị luật an toàn và kết luận hội đồng ràng buộc (Mục 4.3)
Vòng tròn: + tư vấn, + trợ giúp	`ConsensusCriticAgent`, `ResponseAgent`	Tranh biện nhiều vòng, liệt kê (+)/(−), guardrail; câu trả lời có trích dẫn (Mục 4.4)

Ngoài các khối trên bảng, hệ thống bổ sung một **tầng tiếp nhận** (người dùng nhập văn bản tự do hoặc tải bệnh án, hệ thống hỏi lại khi thiếu dữ liệu) và một agent **điều phối** chạy các nhánh song song. Đây là phần cần thiết để sơ đồ trở thành một sản phẩm dùng được.

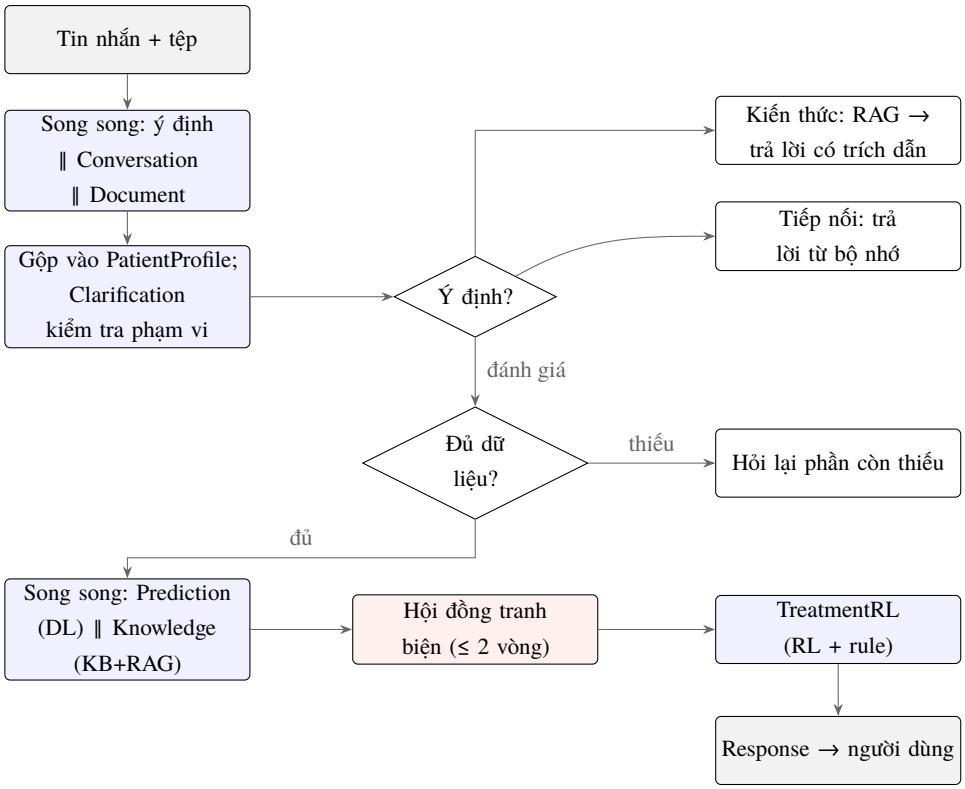
3.2. Vai trò của các agent

Bảng 3.2. Chín agent, thông điệp chúng nhận và kỹ thuật bên trong

Agent	Nhận (`msg_type`)	Vai trò và kỹ thuật
`Orchestrator`	– (khởi tạo mỗi lượt)	Phân loại ý định, lập kế hoạch động, chạy song song bằng `asyncio`
`ConversationAgent`	`EXTRACT_FROM_TEXT`	Bóc tách chỉ số, yếu tố nguy cơ từ tin nhắn tiếng Việt (LLM structured output; dự phòng regex)
`DocumentAgent`	`PARSE_DOCUMENT`	Đọc PDF, ảnh, DOCX, CSV; quy đổi đơn vị (mmol/L → mg/dL…)
`ClarificationAgent`	`SCOPE_CHECK`, `PLAN_QUESTIONS`	Phát hiện hồ sơ ngoài phạm vi sàng lọc (thai phụ, đã chẩn đoán…), đặt tối đa 2 câu hỏi làm rõ
`PredictionAgent`	`PREDICT_RISK`, 4 loại `CHALLENGE_*`	Deep Learning ⇒ Agent: GBDT + MLP trên NHANES và trên BRFSS 3,17 triệu người; giải thích đặc trưng; bảo vệ hoặc nhượng bộ khi bị chất vấn
`KnowledgeAgent`	`RETRIEVE_EVIDENCE`, `ANSWER_QUESTION`, 2 loại `CHALLENGE_*`	KB + RAG: ngưỡng ADA, tìm kiếm lai, GraphRAG trên Neo4j
`ConsensusCriticAgent`	`DELIBERATE`	Hội đồng: phát hiện mâu thuẫn, tranh biện, (+)/(−), phân tầng, Safety Guardrail
`TreatmentRLAgent`	`RECOMMEND_TREATMENT`	RL + rule: Q-learning có ràng buộc an toàn
`ResponseAgent`	`COMPOSE_*`	Viết câu trả lời chỉ từ dữ liệu của các agent, có trích dẫn (LLM; dự phòng mẫu câu)

3.3. Luồng một lượt hội chẩn

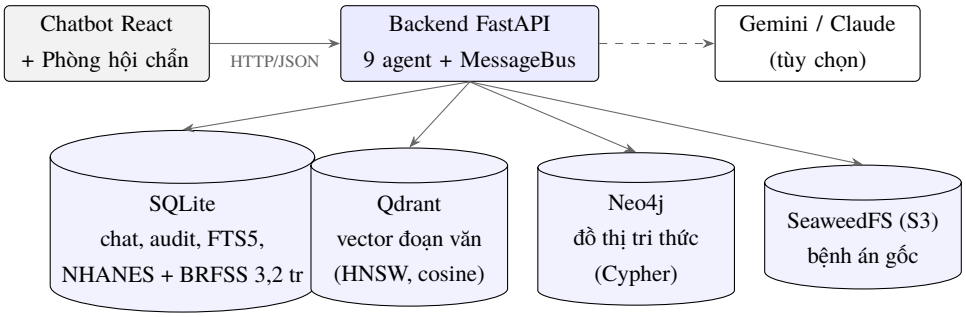
Mỗi tin nhắn của người dùng là một *conversation* trên `MessageBus`. `Orchestrator` lập kế hoạch theo ý định và mức đủ dữ liệu (Hình 3.2): câu hỏi kiến thức chỉ đi nhánh RAG; câu hỏi tiếp nối được trả lời từ bộ nhớ; thiếu dữ liệu thì hỏi lại; đủ dữ liệu thì hai nhánh *Dự đoán* và *Tri thức* chạy song song, rồi đến hội đồng, agent can thiệp và agent phản hồi.



Hình 3.2. Luồng xử lý một lượt chat: Orchestrator rẽ nhánh theo ý định và mức đủ dữ liệu

3.4. Công nghệ và lưu trữ

Bộ lưu trữ được chọn để mỗi loại dữ liệu nằm ở công cụ phù hợp nhất với nó, và để khối "neo4j KB" trên sơ đồ được hiện thực đúng bằng Neo4j (Hình 3.3, Bảng 3.3).



Hình 3.3. Thành phần triển khai và nơi lưu từng loại dữ liệu

Bảng 3.3. Công nghệ sử dụng

Thành phần	Công nghệ	Ghi chú
Điều phối đa agent	Tự xây dựng: `BaseAgent`, `MessageBus`, `asyncio`	Thể hiện rõ cơ chế giao tiếp và tranh biện, không phụ thuộc framework
DB quan hệ	SQLite (WAL) + SQLAlchemy 2	Hồ sơ, tin nhắn, kết quả hội chẩn, audit trail `agent_messages`; bảng `nhanes_samples` (20.646) và `brfss_samples` (3.172.819)
Tìm kiếm toàn văn	SQLite FTS5 [16], BM25	Tokenizer `unicode61 remove_diacritics` khớp tiếng Việt có hoặc không dấu
Vector store	Qdrant [17]	Point id = id của đoạn văn; chỉ mục HNSW [18], khoảng cách cosine
Knowledge Graph	Neo4j 5 [19]	Ràng buộc duy nhất theo tên nút; MERGE theo lô bằng `UNWIND`
Embedding	fastembed, MiniLM đa ngôn ngữ [20]	384 chiều, chạy ONNX trên CPU
ML / DL / RL	scikit-learn, PyTorch, NumPy	GBDT, MLP, Q-learning
LLM (tùy chọn)	Gemini hoặc Claude	Chỉ hiểu và diễn đạt ngôn ngữ; thiếu khóa thì dùng regex và mẫu câu
Giao diện	React 19 + Vite, SVG	Chatbot và Phòng hội chẩn
Triển khai	Docker Compose, Cloudflare Tunnel [21]	`neo4j`, `qdrant`, `seaweedfs`, `backend`, `cloudflared`

CHƯƠNG 4. CÁC THÀNH PHẦN CỦA HỆ THỐNG

4.1. Deep Learning ⇒ Agent

4.1.1. Dữ liệu và bài toán

Khối "DB có sẵn" là bộ dữ liệu NHANES 2011–2018 của CDC [22]: **20.646 người trưởng thành**, trong đó 19,8% mắc ĐTĐ (đã được chẩn đoán, hoặc có FPG ≥ 126 mg/dL, hoặc HbA1c ≥ 6,5%). Dữ liệu được nạp vào bảng `nhanes_samples` của SQLite, và mô hình đọc lại từ đây khi huấn luyện, đúng tinh thần "DB → Deep learning" của sơ đồ. Mô hình chỉ dùng **13 yếu tố nguy cơ**: tuổi, giới, BMI, vòng eo, huyết áp tâm thu và tâm trương, tăng huyết áp, rối loạn mỡ máu, tiền sử gia đình, hút thuốc, vận động, bệnh tim mạch, đột quỵ. Mô hình *không* nhận glucose hay HbA1c. Đây là lựa chọn có chủ đích: xét nghiệm đường huyết là căn cứ chẩn đoán và do `KnowledgeAgent` đối chiếu với ngưỡng ADA, còn mô hình trả lời câu hỏi khác là "với hồ sơ này, nguy cơ mắc ĐTĐ cao đến mức nào". Hai góc nhìn độc lập này là nguyên liệu cho hội đồng kiểm tra chéo.

4.1.2. Mô hình và huấn luyện

Dữ liệu chia 70/10/20 theo phân tầng thành ba phần tách rời: huấn luyện, kiểm định (validation) và kiểm tra (14.451 / 2.065 / 4.130 người; chi tiết cho cả hai bộ dữ liệu ở Bảng 4.3). Hai mô hình cùng được huấn luyện:

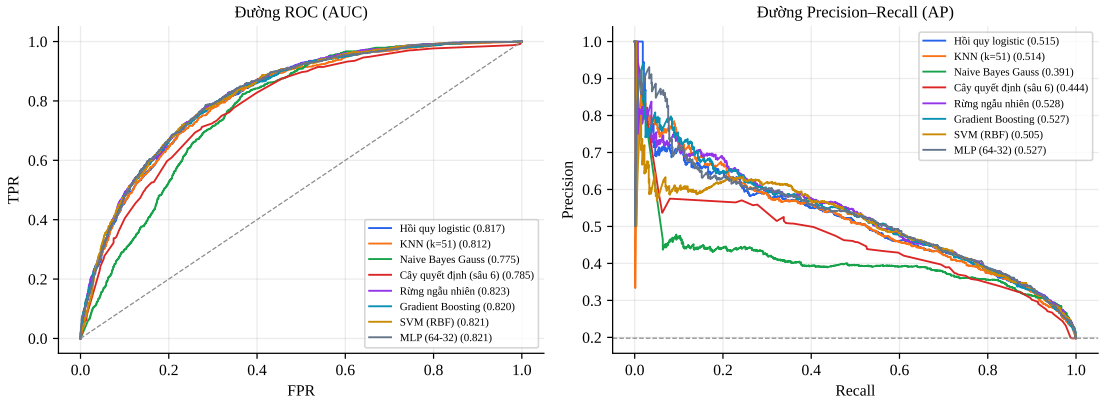
- **ML**: Gradient Boosting dạng histogram (`HistGradientBoostingClassifier`) [23], [24], xử lý giá trị thiếu tự nhiên.
- **DL**: mạng MLP $13 \to 128 \to 64 \to 32 \to 1$, mỗi lớp ẩn có BatchNorm, ReLU, Dropout 0,2; tối ưu AdamW (lr 10^{-3}, weight decay 10^{-4}), batch 512, dừng sớm theo AUC validation (patience 8) [25], [26].

Người dùng thật hiếm khi khai đủ 13 trường. Vì vậy tập huấn luyện được *nhân đôi với bản che ngẫu nhiên*: bản sao bị che 40% các đặc trưng không bắt buộc (mọi trường trừ tuổi và BMI), để mô hình học cách dự đoán khi thiếu dữ liệu.

Bảng 4.1. Kết quả trên tập kiểm tra NHANES (4.130 người)

Mô hình	ROC-AUC	PR-AUC	Brier	AUC khi thiếu dữ liệu
Hồi quy logistic (đối chứng)	0,825	0,533	0,122	0,805
ML: Gradient Boosting	0,817	0,523	0,124	0,802
DL: MLP (PyTorch)	**0,829**	0,537	0,121	**0,809**
Trung bình ML + DL (dùng trong agent)	0,827	**0,539**	0,121	–

Trên nhóm người gốc Á (561 người trong tập kiểm tra), MLP đạt AUC 0,850, cao hơn toàn bộ tập. Hai mô hình đồng ý về phía của ngưỡng ở 92,3% ca. Cần nói thẳng: MLP chỉ nhỉnh hơn hồi quy logistic một chút. Với 13 đặc trưng dạng bảng, học sâu không có lợi thế lớn. Lý do vẫn dùng cả hai là để *kiểm tra chéo*: khi ML và DL lệch nhau nhiều, đó thường là dấu hiệu hồ sơ bất thường, và hội đồng sẽ chất vấn (Mục 4.4).



Hình 4.1. Đường ROC và Precision–Recall của các mô hình trên tập kiểm tra NHANES

4.1.3. Dữ liệu quy mô lớn: BRFSS 2011–2023

Để mô hình học từ một quần thể đủ lớn và đa dạng, khối DB được bổ sung **BRFSS** (Behavioral Risk Factor Surveillance System), khảo sát sức khỏe qua điện thoại lớn nhất thế giới do CDC thực hiện hằng năm. Bản BRFSS 2015 đã làm sạch (253.680 dòng, một năm) là bộ dữ liệu phổ biến cho bài toán dự đoán ĐTĐ. Project không dùng bản đó mà tự xử lý **dữ liệu gốc của bảy năm lẻ 2011–2023** bằng `scripts/fetch_brfss.py`. Chỉ năm lẻ được dùng vì câu hỏi về tăng huyết áp và cholesterol chỉ có trong các năm này. Kết quả là **3.172.819 người**, gấp 12,5 lần bản phổ biến (Bảng 4.2).

Bảng 4.2. Dữ liệu BRFSS theo năm sau khi chuẩn hóa

Năm	Số người	ĐTĐ	Tiền ĐTĐ
2011	504.530	12,4%	1,6%
2013	490.942	12,7%	1,8%
2015	440.658	13,0%	1,7%
2017	449.219	13,5%	1,8%
2019	417.423	13,8%	2,2%
2021	437.708	13,2%	2,3%
2023	432.339	13,8%	2,5%
Tổng	**3.172.819**	13,2%	2,0%

Tên biến của BRFSS thay đổi qua các năm (`DIABETE2/3/4`, `_RFCHOL/1/2/3`, `SEX/SEXVAR`…). Script dò biến theo danh sách ứng viên cho từng năm và suy ra biến còn thiếu (năm 2011 chưa có `_MICHD`, nên bệnh tim được suy từ nhồi máu cơ tim và bệnh mạch vành). Mười một đặc trưng được đặt đúng tên trường của hồ sơ bệnh nhân để agent ánh xạ trực tiếp: tuổi, giới, BMI, tăng huyết áp, rối loạn mỡ máu, hút thuốc, vận động, bệnh tim, đột quỵ, uống nhiều rượu bia, sức khỏe tự đánh giá. Nhãn ĐTĐ là *tự khai* (“bác sĩ từng nói bạn bị tiểu đường”, không tính ĐTĐ thai kỳ); câu trả lời “không biết” bị loại. Toàn bộ 3,17 triệu dòng được nạp vào bảng `brfss_samples` của SQLite rồi huấn luyện cùng quy trình với NHANES, MLP dùng batch 4.096 và dừng sớm sau 10 epoch. Toàn bộ 3.172.819 dòng đều được dùng, nhưng mỗi phần một vai trò riêng (Bảng 4.3).

Bảng 4.3. Chia dữ liệu huấn luyện – kiểm định – kiểm tra (70/10/20, phân tầng theo nhãn, `random_state=42`)

Phần	NHANES	BRFSS	Vai trò
Huấn luyện (70%)	14.451	2.220.973	Mô hình học trực tiếp
sau tăng cường (×2)	28.902	4.441.946	Thêm bản sao bị che 40% đặc trưng không bắt buộc, để học cách dự đoán khi người dùng khai thiếu
Kiểm định (10%)	2.065	317.282	Chọn epoch, giảm tốc độ học, dừng sớm MLP; không dùng để cập nhật trọng số
Kiểm tra (20%)	4.130	634.564	Chỉ dùng một lần để báo cáo AUC, PR-AUC, Brier (Bảng 4.1, Bảng 4.4)
Tổng	**20.646**	**3.172.819**	

Tập kiểm tra được giữ tách riêng từ đầu đến cuối, nên các chỉ số trong báo cáo phản ánh khả năng dự đoán trên người mà mô hình chưa từng thấy. Tập kiểm tra bị che ("AUC khi thiếu dữ liệu") được tạo từ chính tập kiểm tra này. Riêng phần đánh giá đầu–cuối của hệ đa agent (Chương 5) lấy 400 ca từ tập kiểm tra NHANES.

Bảng 4.4. Kết quả trên tập kiểm tra BRFSS (634.564 người)

Mô hình	ROC-AUC	PR-AUC	AUC khi thiếu dữ liệu	AUC người gốc Á
Hồi quy logistic (đối chứng)	0,825	0,399	0,804	0,843
ML: Gradient Boosting	**0,833**	**0,422**	**0,813**	**0,849**
DL: MLP (PyTorch)	**0,833**	0,422	**0,813**	0,848

Nhận xét. (i) Trên 634.564 người kiểm tra, cả GBDT và MLP đạt AUC 0,833 và vượt hồi quy logistic rõ hơn so với trên NHANES. Với dữ liệu lớn, mô hình phi tuyến có đủ mẫu để học tương tác giữa các yếu tố. (ii) Nhóm người gốc Á có 14.249 người trong tập kiểm tra, gấp 25 lần NHANES (561), và AUC đạt 0,849. Đây là căn cứ tốt hơn nhiều cho câu hỏi "mô hình có dùng được cho người châu Á không". (iii) Dù gấp 150 lần dữ liệu, AUC chỉ ngang NHANES (0,83). Giới hạn nằm ở *thông tin* trong các đặc trưng tự khai, không nằm ở số mẫu. Vì vậy NHANES, với nhãn xác nhận bằng xét nghiệm và số đo thật (vòng eo, huyết áp), vẫn là mô hình chính, còn BRFSS là ý kiến thứ hai.

4.1.4. Từ mô hình thành agent

Một con số xác suất chưa đủ để tư vấn. `PredictionAgent` bọc hai mô hình thành một agent có bốn năng lực:

1. **Dự đoán**: xác suất trung bình của ML và DL, kèm *nguy cơ tương đối* so với tỉ lệ nền 19,8% của quần thể.
2. **Giải thích**: tác động của từng đặc trưng, tính bằng cách thay giá trị đó bằng giá trị tham chiếu rồi đo xác suất thay đổi bao nhiêu.
3. **Tự đánh giá**: độ đầy đủ của dữ liệu (tỉ lệ đặc trưng có giá trị thật) và độ lệch giữa hai mô hình.
4. **Tranh biện**: trả lời bốn loại `CHALLENGE` (chất lượng dữ liệu, bất đồng ML–DL, mâu thuẫn với xét nghiệm, bất đồng giữa hai quần thể dữ liệu) bằng cách bảo vệ, nhượng bộ hoặc hạ độ tin cậy.
5. **Ý kiến thứ hai quy mô lớn**: chạy thêm cặp GBDT + MLP của BRFSS. Hai bộ dữ liệu có tỷ lệ nền khác nhau (19,8% và 13,2%), nên hai mô hình được so theo *nguy cơ tương đối* (gấp bao nhiêu lần trung bình quần thể của mình), không so xác suất tuyệt đối.

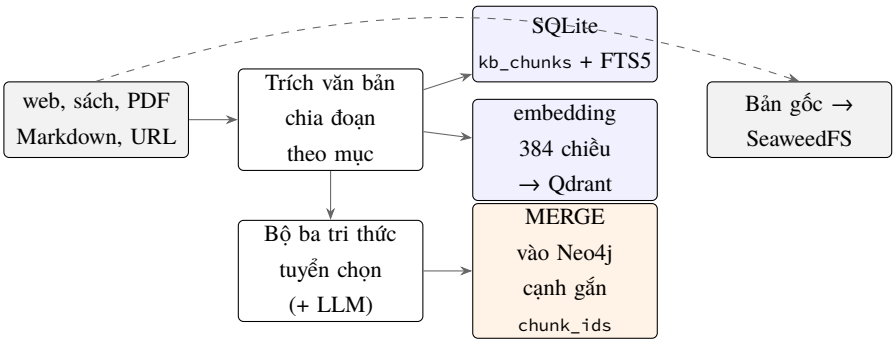
Ngoài ra agent chạy mạng hồi quy (RNN) khi người dùng tải chuỗi đo glucose ≥ 5 điểm, để tính các chỉ số theo dõi liên tục (TIR, TBR, CV). Agent không sinh dữ liệu giả khi thiếu đầu vào.

Phân loại. Kết quả dự đoán cùng với xét nghiệm được hội đồng phân tầng thành bốn mức: *Rất cao* (xét nghiệm ở ngưỡng ĐTĐ), *Cao* (tiền ĐTĐ hoặc xác suất ≥ 30%), *Trung bình* (15–30%), *Thấp*. Xét nghiệm là căn cứ chính, mô hình bổ sung nguy cơ dài hạn.

4.2. Knowledge Base và RAG

4.2.1. Nguồn tri thức và quy trình tạo KB

Khối "web, book → tạo → neo4j KB" được hiện thực bằng `scripts/ingest_kb.py`. Nguồn là 10 tài liệu tiếng Việt biên soạn từ hướng dẫn ADA 2024 [1] và QĐ 5481/QĐ-BYT [3]: tiêu chuẩn chẩn đoán, tiền ĐTĐ, mục tiêu điều trị, sàng lọc người Việt, triệu chứng cảnh báo, biến chứng, thuốc, lối sống và dinh dưỡng, theo dõi glucose liên tục. Có thể nạp thêm bất kỳ trang web hay PDF nào bằng `--url`. Quy trình:



Hình 4.2. Quy trình tạo Knowledge Base từ web và sách

Kết quả là 36 đoạn văn và một đồ thị tri thức **34 nút, 40 cạnh** trên Neo4j. Ontology gồm 6 nhãn nút (`RiskFactor`, `Disease`, `Complication`, `Test`, `Drug`, `Intervention`) và 7 loại quan hệ (`HAS_RISK_FACTOR`, `DIAGNOSED_BY`, `CAUSES`, `SCREENED_BY`, `TREATED_BY`, `CONTRAINDICATED_IN`, `PREVENTED_BY`). Mỗi cạnh lưu đoạn trích làm bằng chứng và `chunk_ids` trỏ về đoạn văn gốc, nên khi GraphRAG đi qua một cạnh, hệ thống trích dẫn được đúng đoạn tài liệu. Quá trình nạp chạy lại an toàn: tài liệu không đổi (cùng mã băm và cùng mô hình embedding, và đã có trong Qdrant) sẽ được bỏ qua.



Hình 4.3. Đồ thị tri thức về ĐTĐ (nút theo nhãn của ontology)

4.2.2. Truy xuất: tìm kiếm lai và GraphRAG

`KnowledgeAgent` kết hợp ba cách truy xuất [27], [28]:

- **Vector** trên Qdrant: bắt các câu hỏi diễn đạt khác từ ngữ trong tài liệu (“khát nước

nhiều" và "uống nhiều").

- **Toàn văn** BM25 [29] trên SQLite FTS5: bắt đúng từ khóa chuyên môn (HbA1c, SGLT2, eGFR). Câu hỏi tự nhiên được nối từ bằng `OR`, bỏ các từ hư ("bao nhiêu", "thì", "là"), và tiêu đề mục có trọng số gấp 3 nội dung.
- **GraphRAG** [30] trên Neo4j: nhận diện thực thể trong hồ sơ hoặc câu hỏi, duyệt lân cận 1–2 bước bằng Cypher, lấy các đoạn văn gắn với cạnh.

Hai danh sách vector và toàn văn được gộp bằng Reciprocal Rank Fusion [31]: $\text{score}(d) = \sum_r \frac{1}{60+\text{rank}_r(d)}$. Ngoài truy xuất văn bản, agent còn đối chiếu xét nghiệm của bệnh nhân với ngưỡng ADA (FPG, HbA1c, OGTT, glucose bất kỳ) và kiểm tra quy tắc "hai kết quả bất thường" để biết đã đủ tiêu chí chẩn đoán hay cần xét nghiệm lại.

Đoạn mã 4.1. GraphRAG trên Neo4j (`src/storage/graph_store.py`, rút gọn)

```
def neighborhood(self, start_names, max_hops=2, limit=60):
    records = self._run(f"MATCH p = (s)-[*1..{int(max_hops)}]-(t) WHERE s.name IN $names "
                        f"RETURN p LIMIT $lim", names=list(start_names), lim=limit * 4)
    edges = {}
    for rec in records:
        for hop, rel in enumerate(rec["p"].relationships, 1):   # cạnh thứ hop trên đường đi
            if rel.element_id in edges:
                edges[rel.element_id]["hops"] = min(edges[rel.element_id]["hops"], hop)
                continue
            edges[rel.element_id] = {
                "head": rel.start_node["name"], "relation": rel.type, "tail":
                 ↪  rel.end_node["name"],
                "evidence": rel.get("evidence", ""), "chunk_ids": list(rel.get("chunk_ids")
                 ↪  or []),
                "hops": hop}
    return sorted(edges.values(), key=lambda e: e["hops"])[:limit]
```

4.2.3. Đánh giá truy xuất

Bộ đánh giá gồm 60 câu hỏi tiếng Việt có đáp án (đoạn văn đúng được gắn tay), chia bốn loại: từ khóa (20), diễn đạt lại (19), con số (17), nhiều bước (4). Bảng 4.5 đo trên bộ lưu trữ hiện tại của project.

Bảng 4.5. Truy xuất trên 60 câu hỏi tiếng Việt (36 đoạn văn)

Phương pháp	hit@1	hit@3	hit@5	MRR@10
Vector (Qdrant, MiniLM)	0,567	0,750	0,833	0,675
Toàn văn (SQLite FTS5, BM25)	**0,833**	**0,967**	**0,983**	**0,899**
Lai RRF (Qdrant + FTS5, đang dùng)	0,717	0,900	0,933	0,820
Tham chiếu: lai pgvector + tsvector (bản trước)	0,667	0,850	0,917	0,774

Nhận xét. (i) Bộ lưu trữ mới tốt hơn bản PostgreSQL trước đó (hit@3 0,90 so với 0,85), chủ yếu nhờ BM25 của FTS5 cùng xử lý từ hư và trọng số tiêu đề. Độ trễ trung vị của một truy vấn lai là 2,8 ms. (ii) Trên bộ câu hỏi này, toàn văn đơn lẻ lại tốt nhất. Nguyên nhân là mô hình embedding MiniLM yếu với tiếng Việt chuyên ngành (hit@3 0,75), nên khi gộp, nó kéo xếp hạng của BM25 xuống. Phần thí nghiệm của tiểu luận cho thấy mô hình E5 đa ngôn ngữ đạt hit@3 0,98 trên cùng bộ câu hỏi. Đổi embedding hoặc gán trọng số cho RRF là hướng cải thiện trực tiếp. Hệ thống vẫn giữ tìm kiếm lai vì bộ 60 câu nhỏ và có nhiều câu chứa đúng từ khóa của tài liệu, trong khi người dùng thật thường diễn đạt tự do hơn.

4.3. Reinforcement Learning và luật

4.3.1. Agent gợi ý can thiệp bằng Q-learning

`TreatmentRLAgent` mô hình hóa việc chọn bước can thiệp như một quá trình quyết định Markov (MDP). Trạng thái là tổ hợp 4 mức glucose × 3 mức BMI × 2 mức bệnh đồng mắc, tức 24 trạng thái. Hành động gồm 4 lựa chọn: (0) thay đổi lối sống, (1) metformin, (2) phối hợp thuốc có lợi ích tim–thận, (3) khám chuyên khoa hoặc insulin. Q-learning [32], [33] cập nhật giá trị hành động theo sai số thời gian:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big], \qquad \alpha = 0{,}1,\ \gamma = 0{,}9, \quad (4.1)$$

với chính sách khám phá ϵ-tham lam ($\epsilon = 0{,}15$), huấn luyện 1.500 tập. Hàm thưởng mô phỏng mức cải thiện đường huyết trừ chi phí và độ xâm lấn: với người glucose bình thường, lối sống được thưởng +4, thuốc bị phạt; với người ở ngưỡng ĐTĐ có bệnh đồng mắc, phối hợp tim–thận được thưởng cao nhất; với đường huyết rất cao, chuyển chuyên khoa được +5.

Hạn chế cần nói rõ. Môi trường và hàm thưởng do nhóm tự đặt theo khuyến cáo, không học từ dữ liệu điều trị thật. Q-learning ở đây chủ yếu minh họa cơ chế "agent học ra quyết định tuần tự" trong sơ đồ, và chính sách học được không bao giờ được hành động một mình.

4.3.2. Luật ràng buộc chính sách học được

Khối "rule" trên sơ đồ là lớp luật đứng trên chính sách RL. Luật luôn thắng khi hai bên khác nhau:

- Hội đồng *chưa* xác định ĐTĐ → hành động bị giới hạn ở thay đổi lối sống.
- Chẩn đoán cần xác nhận lại (mới có một xét nghiệm bất thường) → hoãn metformin.
- Guardrail khẩn được kích hoạt → chuyển khám chuyên khoa.
- Chống chỉ định trong đồ thị tri thức (metformin khi eGFR < 30) → luôn được nêu ra.

Agent trả về cả hành động RL gốc (`rl_action_id`), hành động sau ràng buộc và cờ `overridden_by_consensus`, nên người xem thấy được khi nào luật đã ghi đè mô hình học.

Bảng 4.6. Safety Guardrail: luật an toàn luôn được ưu tiên

Mức	Điều kiện	Hành động
Cấp cứu	Glucose ≥ 600 mg/dL	Đến cơ sở y tế cấp cứu ngay
Khẩn	Glucose ≥ 300 mg/dL hoặc HbA1c ≥ 10%	Khám chuyên khoa Nội tiết sớm; RL chuyển sang hành động 3
Khẩn	Chuỗi đo có glucose < 54 mg/dL	Cảnh báo hạ đường huyết mức 2
Ưu tiên	BMI ≥ 25 + tăng huyết áp + tiền sử gia đình hoặc tim mạch	Nhóm nguy cơ "đỏ": ưu tiên xét nghiệm và kiểm soát yếu tố tim mạch

4.4. Hội đồng tranh biện: tư vấn và trợ giúp

Vòng tròn lớn trên sơ đồ là `ConsensusCriticAgent`. Nó nhận kết quả của hai nhánh, nhưng không chỉ tổng hợp: khi phát hiện mâu thuẫn, nó gửi `CHALLENGE` qua bus tới agent liên quan, nhận `RESPONSE` và cập nhật kết luận. Phản hồi ở vòng trước có thể mở ra câu hỏi ở vòng sau, tối đa 2 vòng.

Bảng 4.7. Giao thức tranh biện của hội đồng

Vấn đề phát hiện	Chất vấn	Phản hồi và ảnh hưởng
Dữ liệu thật < 35% số đặc trưng	Prediction · `CHALLENGE_DATA_QUALITY`	Hạ độ tin cậy, liệt kê đặc trưng còn thiếu → (−)
ML và DL lệch ≥ 15 điểm %	Prediction · `CHALLENGE_MODEL_DISAGREEMENT`	Chỉ ra đặc trưng hai mô hình đánh giá khác nhau nhất → (−)
ML ≥ 30% nhưng xét nghiệm bình thường	Prediction · `CHALLENGE_ML_VS_LABS`	Giữ quan điểm "nguy cơ dài hạn" → (−) và **mở vòng 2**
(vòng 2) ML giữ quan điểm	Knowledge · `CHALLENGE_SCREENING_PLAN`	Lịch tầm soát ADA có trích dẫn → (+)
ML < 15% nhưng xét nghiệm ở ngưỡng ĐTĐ	Prediction · `CHALLENGE_ML_VS_LABS`	Nhượng bộ cho xét nghiệm → (−)
Xét nghiệm ở ngưỡng ĐTĐ	Knowledge · `CHALLENGE_VERIFY_CRITERIA`	Kiểm tra quy tắc "2 kết quả bất thường" → "đủ tiêu chí" hoặc "cần xét nghiệm lại"
Mô hình NHANES và BRFSS ở hai phía ngược nhau (một bên ≥ 1,5 lần, bên kia ≤ 0,8 lần trung bình)	Prediction · `CHALLENGE_COHORT_DISAGREEMENT`	Giải trình khác biệt về nhãn và đặc trưng của hai quần thể, giữ mô hình chính, hạ độ tin cậy → (−). Cùng phía thì ghi (+) "mô hình quy mô lớn đồng thuận"

Đoạn mã 4.2. Vòng tranh biện của hội đồng (`src/agents/consensus_critic_agent.py`, rút gọn)

```
for round_no in range(1, settings.max_debate_rounds + 1):
    issues = [i for i in _detect_issues(state) if i[0] not in raised]
    if not issues:
        break                                          # không còn mâu thuẫn → kết thúc
    raised.update(i[0] for i in issues)
    replies = await asyncio.gather(*(                  # chất vấn song song qua MessageBus
        self.ask(receiver, msg_type, payload, msg.conversation_id, performative="CHALLENGE")
        for _, receiver, msg_type, payload in issues))
    for (issue_id, receiver, msg_type, _), reply in zip(issues, replies):
        debate.append({"round": round_no, "issue": issue_id, "challenged": receiver, ...})
        if reply.performative == "RESPONSE":
            _apply_response(state, issue_id, reply.payload)   # cập nhật (+)/(-), độ tin cậy
return self.reply(msg, "CONSENSUS_RESULT", _conclude(state, profile, debate))
```

Kết luận của hội đồng gồm mức nguy cơ, danh sách bằng chứng đồng thuận (+), điểm mâu thuẫn hoặc cần lưu ý (−), cảnh báo an toàn, và kế hoạch tầm soát nếu có. `ResponseAgent` dùng đúng các dữ liệu đó để viết câu trả lời: phần *tư vấn* (mức nguy cơ và lý do, khuyến cáo có trích dẫn nguồn) và phần *trợ giúp* (bước tiếp theo nên làm, câu hỏi làm rõ khi thiếu dữ liệu). Agent bị cấm thêm thông tin không có trong kết quả của các agent khác, và luôn nhắc rằng hệ thống không thay thế bác sĩ.

4.5. Tiếp nhận, làm rõ và bộ nhớ

Ba agent ở tầng tiếp nhận biến dữ liệu thô thành hồ sơ có cấu trúc:

- `ConversationAgent` bóc tách chỉ số từ câu tự do, ví dụ "mẹ tôi 61 tuổi, nặng 68 ký cao 1m52, sáng nay thử đường máu lúc chưa ăn được 6,4". Đơn vị không ghi rõ được suy ra (6,4 → mmol/L → 115 mg/dL) kèm cờ `unit_inferred`.
- `DocumentAgent` đọc phiếu xét nghiệm dạng PDF, ảnh chụp, DOCX hoặc CSV chuỗi đo glucose.
- `ClarificationAgent` phát hiện hồ sơ ngoài phạm vi sàng lọc (thai phụ, người đã được chẩn đoán, đang dùng thuốc hạ đường huyết, HbA1c lệch nhiều so với glucose…) và đặt tối đa 2 câu hỏi làm rõ mỗi lượt, không hỏi lại câu đã hỏi.

Mỗi trường trong `PatientProfile` lưu giá trị, nguồn (`text`, `file:<tên>`), độ tin cậy, đoạn trích làm bằng chứng và cờ cảnh báo. Hệ thống không tự điền giá trị mặc định cho chỉ số lâm sàng. Khi hai nguồn mâu thuẫn, hội đồng báo (−) để người dùng xác nhận. Bộ nhớ hai tầng (hồ sơ tích lũy và 10 tin nhắn gần nhất) giúp hệ thống hiểu câu trả lời cụt ("62" sau khi được hỏi tuổi) và trả lời câu hỏi tiếp nối mà không chạy lại hội đồng.

CHƯƠNG 5. CODE VÀ DEMO

5.1. Cấu trúc mã nguồn

Đoạn mã 5.1. Cấu trúc thư mục chính của project

```
MultiAgent/
├── app.py                          FastAPI: API + phục vụ giao diện
├── docker-compose.yml              neo4j, qdrant, seaweedfs, backend, cloudflared
├── src/
│   ├── communication/
│   │   ├── message_bus.py          AgentMessage, BaseAgent, MessageBus (định tuyến + audit
↪  trail)
│   │   └── summary.py              tóm tắt một dòng cho mỗi thông điệp (cho Phòng hội chẩn)
│   ├── agents/                   9 agent: orchestrator, conversation, document, clarification,
│   │                               prediction, knowledge, consensus_critic, treatment_rl,
↪  response
│   ├── models/                     tabular_ml (GBDT), deep_mlp (PyTorch), rnn_temporal,
↪  qlearning_treatment
│   ├── knowledge/                  ingest (tạo KB), embedder, clinical_criteria (ngưỡng ADA),
↪  scope
│   ├── storage/
│   │   ├── db.py, models.py        SQLite (SQLAlchemy) + bảng ảo FTS5
│   │   ├── vector_store.py         QdrantVectorStore: tìm kiếm lai Qdrant + FTS5, RRF
│   │   └── graph_store.py          Neo4jGraphStore: MERGE bộ ba, duyệt lân cận bằng Cypher
│   └── api/                        routes_chat, routes_session, routes_system, routes_lab
├── scripts/                        fetch_nhanes, fetch_brfss (3,17 triệu người), train_model,
│                                   ingest_kb, chat_cli
├── data/kb_sources/                10 tài liệu ADA 2024 / QĐ 5481 + curated_triples.json
├── frontend/src/components/        MainChat, ConsultationDetails, AgentRoom (Phòng hội chẩn)
└── tests/                          43 kiểm thử tự động
```

5.2. Các hàm chính

Bảng 5.1. Các hàm và lớp chính

Hàm / lớp	Vai trò
`MessageBus.send`	Ghi thông điệp, định tuyến tới agent nhận, chuyển ngoại lệ thành `FAILURE`, đo thời gian xử lý
`OrchestratorAgent.run_turn`	Phân loại ý định, lập kế hoạch, chạy song song các nhánh, trả về câu trả lời và transcript
`PredictionAgent.handle`	Dự đoán bằng GBDT + MLP; trả lời ba loại chất vấn
`QdrantVectorStore.hybrid_search`	Truy vấn Qdrant và FTS5, gộp bằng RRF, đọc nội dung đoạn từ SQLite
`Neo4jGraphStore.neighborhood`	GraphRAG: duyệt lân cận 1–2 bước từ các thực thể, trả về cạnh kèm `chunk_ids`
`ConsensusCriticAgent.handle`	Phát hiện mâu thuẫn, tranh biện nhiều vòng, phân tầng, guardrail
`TreatmentRLAgent.handle`	Chọn hành động theo Q-table rồi áp luật ràng buộc
`ingest_documents`, `rebuild_graph`	Tạo KB: SQLite + Qdrant + Neo4j
`AgentRoom` (React)	Phát lại transcript trên sơ đồ agent

Đoạn mã 5.2. MessageBus: các agent không gọi hàm của nhau (`src/communication/message_bus.py`, rút gọn)

```python
async def send(self, msg: AgentMessage) -> AgentMessage:
    self._record(msg)                                    # audit trail theo conversation
    agent = self.agents.get(msg.receiver)
    if agent is None or msg.msg_type not in agent.handles:
        reply = failure(msg, "NOT_UNDERSTOOD")
    else:
        try:
            reply = await agent.handle(msg)
        except Exception as exc:                         # lỗi một agent không làm sập hội đồng
            reply = agent.reply(msg, f"{msg.msg_type}_FAILED", {"error": str(exc)},
                                performative="FAILURE")
    reply.elapsed_ms = int((time.perf_counter() - started) * 1000)
    self._record(reply)
    return reply
```

Đoạn mã 5.3. Tìm kiếm lai Qdrant + SQLite FTS5 (`src/storage/vector_store.py`, rút gọn)

```
def _fts_ranked(self, query_text, limit):
    words = _words(query_text)                          # bỏ dấu câu và từ hư tiếng Việt
    match = " OR ".join(f'"{w}"' for w in words)
    return self.session.execute(text(
        "SELECT rowid, -bm25(kb_chunks_fts, 3.0, 1.0) FROM kb_chunks_fts "
        "WHERE kb_chunks_fts MATCH :q ORDER BY bm25(kb_chunks_fts, 3.0, 1.0) LIMIT :n"),
        {"q": match, "n": limit}).all()

def hybrid_search(self, query_text, query_vec, k=5, pool=30):
    vec = self.client.query_points(self.collection, query=query_vec, limit=pool).points
    fused = _rrf([(p.id, p.score) for p in vec], self._fts_ranked(query_text, pool), k + 5)
    return self._attach(fused)[:k]                      # nối nội dung đoạn văn từ SQLite
```

5.3. Cách chạy

Đoạn mã 5.4. Cài đặt và chạy hệ thống

```
cp .env.example .env                                    # GEMINI_API_KEY không bắt buộc
docker compose up -d neo4j qdrant seaweedfs             # SQLite là file, không cần container
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python scripts/fetch_nhanes.py                # tải NHANES vào SQLite
.venv/bin/python scripts/fetch_brfss.py                 # BRFSS 2011-2023: 3.172.819 người
↪  (~740 MB)
.venv/bin/python scripts/train_model.py                 # huấn luyện GBDT + MLP trên NHANES và
↪  BRFSS
.venv/bin/python scripts/ingest_kb.py                   # tạo KB: SQLite + Qdrant + Neo4j
.venv/bin/python -m uvicorn app:app --port 8000         # http://localhost:8000
.venv/bin/python -m pytest -q                           # 43 kiểm thử
docker compose --profile app up -d --build             # triển khai toàn bộ + Cloudflare Tunnel
```

Khi chạy, có thể xem đồ thị tri thức trực tiếp trên Neo4j Browser (`http://localhost:7474`) và các vector trên Qdrant Dashboard (`http://localhost:6333/dashboard`).

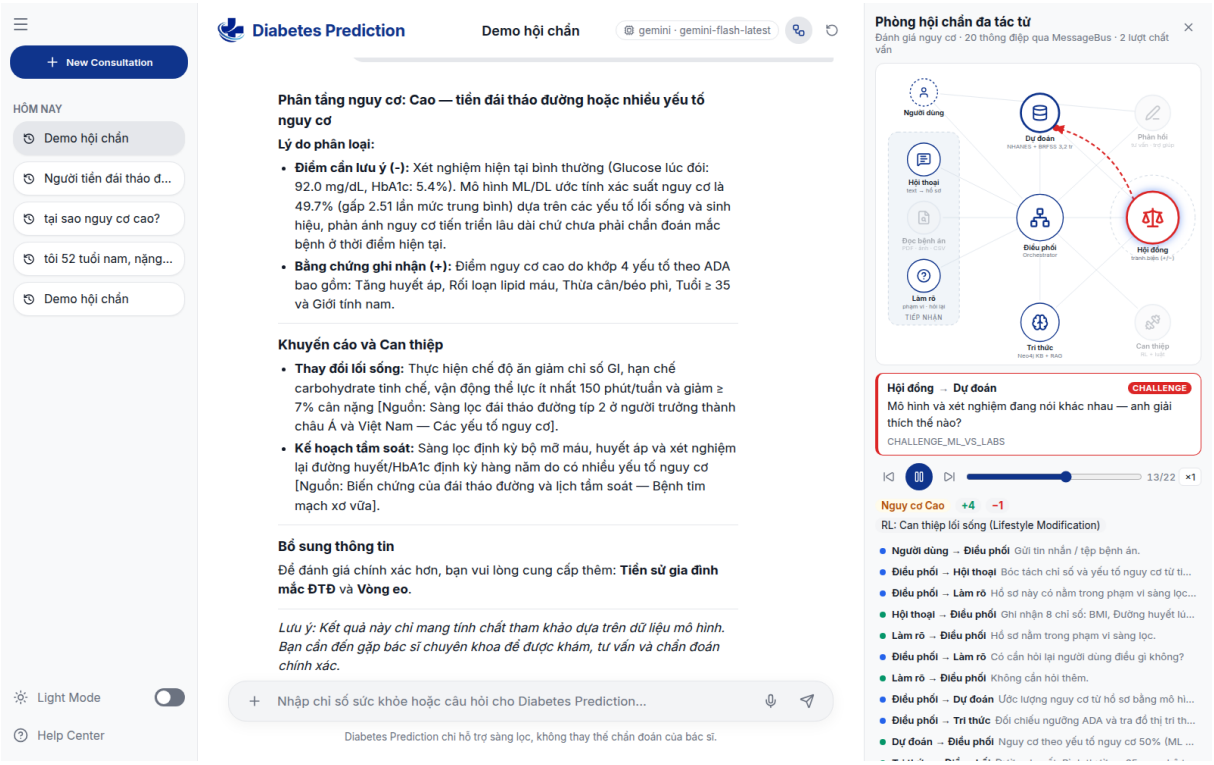
5.4. Demo: chatbot và Phòng hội chẩn

Giao diện là một chatbot web (React). Người dùng nhập tin nhắn tự do, nói bằng giọng, hoặc đính kèm phiếu xét nghiệm. Để người xem *thấy* được hệ đa agent đang làm việc, giao diện có thêm panel **Phòng hội chẩn** bên phải (Hình 5.1):

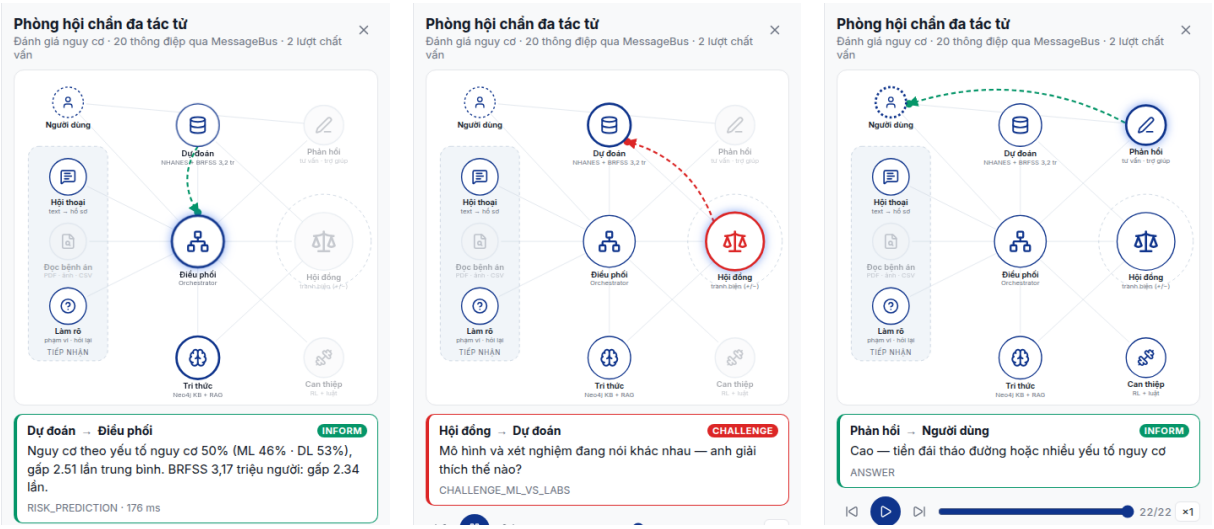
- **Sơ đồ agent** bố trí theo đúng sơ đồ của đề bài: Dự đoán (DB → ML + DL) ở trên, Tri thức (Neo4j KB + RAG) ở dưới, Hội đồng (+/−) bên phải, Can thiệp (RL + luật) ở góc dưới, tầng tiếp nhận bên trái, Điều phối ở giữa.
- **Phát lại từng thông điệp**: mỗi bước vẽ một mũi tên từ agent gửi tới agent nhận,

có chấm sáng chạy dọc theo cung, màu theo performative (xanh dương `REQUEST`, xanh lá `INFORM`, đỏ `CHALLENGE`, cam `RESPONSE`). Agent đang nói được viền sáng; agent chưa tham gia bị làm mờ.

- **Bong bóng phát biểu**: câu tóm tắt nội dung thông điệp, ví dụ "Mô hình và xét nghiệm đang nói khác nhau — anh giải thích thế nào?".
- **Điều khiển**: phát, tạm dừng, tua từng bước, thanh trượt, đổi tốc độ; danh sách toàn bộ thông điệp bấm được để nhảy tới.
- Mỗi câu trả lời trong khung chat có nút "Xem các tác tử phối hợp" để mở lại đúng lượt hội chẩn đó.



Hình 5.1. Giao diện chatbot và Phòng hội chẩn: hội đồng đang chất vấn agent Dự đoán (mũi tên đỏ)



(a) Nhánh DL trả kết quả (b) Hội đồng chất vấn (c) Trả lời người dùng

Hình 5.2. Ba thời điểm khi phát lại một lượt hội chẩn trong Phòng hội chẩn

5.5. Ví dụ một lượt hội chẩn

Tin nhắn demo: "Tôi nam 58 tuổi, BMI 31, tăng huyết áp, mỡ máu cao, đường huyết lúc đói 92 mg/dL, HbA1c 5.4%. Nguy cơ của tôi thế nào?". Đây là ca điển hình gây mâu thuẫn: xét nghiệm *bình thường* nhưng yếu tố nguy cơ *dày đặc*. Hệ thống chạy ở chế độ không LLM. Bảng 5.2 là toàn bộ 22 thông điệp mà MessageBus ghi lại, kèm câu tóm tắt hiển thị trong Phòng hội chẩn.

Bảng 5.2. Luồng 22 thông điệp của lượt hội chẩn demo (ms: thời gian xử lý ở thông điệp trả lời)

#	Gửi	Nhận	Perf.	Nội dung (tóm tắt)	ms
1–2	Orchestrator ⇄	Conversation	REQ / INF	Ghi nhận 6 chỉ số: FPG, HbA1c, BMI, tuổi, tăng huyết áp, rối loạn mỡ máu	3
3–6	Orchestrator ⇄	Clarification	REQ / INF	Hồ sơ nằm trong phạm vi sàng lọc; không cần hỏi thêm	0
7, 9	Orchestrator ⇄	Prediction	REQ / INF	NHANES: nguy cơ 47% (ML 45% · DL 50%), gấp 2,38 lần trung bình; BRFSS 3,17 triệu người: gấp 2,43 lần	232
8, 10	Orchestrator ⇄	Knowledge	REQ / INF	Đường huyết bình thường; 25 quan hệ trên đồ thị tri thức	1.260
11	Orchestrator	Critic	REQUEST	Mở hội đồng: tổng hợp bằng chứng và tranh biện	
12	Critic	Prediction	**CHALLENGE**	Hồ sơ còn thiếu nhiều đặc trưng — dự đoán còn tin được không?	
13	Prediction	Critic	RESPONSE	Mới có 31% đặc trưng; mô hình được huấn luyện cho dữ liệu thiếu, hạ độ tin cậy	0
14	Critic	Prediction	**CHALLENGE**	Mô hình và xét nghiệm nói khác nhau — giải thích thế nào?	
15	Prediction	Critic	RESPONSE	Mô hình không dùng glucose nên phản ánh nguy cơ tiến triển; đồng ý xét nghiệm là căn cứ chẩn đoán, đề nghị giữ tầm soát	0
16	Critic	Knowledge	**CHALLENGE**	(vòng 2) Xét nghiệm bình thường nhưng nguy cơ dài hạn cao — tầm soát thế nào?	
17	Knowledge	Critic	RESPONSE	ADA: sàng lọc lại ít nhất mỗi 3 năm, sớm hơn khi nhiều yếu tố nguy cơ; giảm cân, vận động 150 phút/tuần	22
18	Critic	Orchestrator	INFORM	Kết luận: Cao — 5 (+) / 2 (−), 2 vòng tranh biện; có (+) "mô hình quy mô lớn đồng thuận"	23
19–20	Orchestrator ⇄	TreatmentRL	REQ / INF	Đề xuất: can thiệp lối sống (luật giới hạn vì xét nghiệm chưa ở ngưỡng ĐTĐ)	0
21–22	Orchestrator ⇄	Response	REQ / INF	Viết câu trả lời có trích dẫn	0

Ví dụ này cho thấy giá trị của tranh biện. Một hệ thống chỉ dùng ngưỡng sẽ trả lời "đường huyết bình thường". Một hệ thống chỉ dùng mô hình sẽ trả lời "nguy cơ 47%" mà không giải thích vì sao xét nghiệm lại bình thường. Hội đồng đưa cả hai góc nhìn ra, hòa giải chúng thành một kế hoạch tầm soát có trích dẫn, và cho người dùng thấy cả điểm đồng thuận lẫn điểm còn cần lưu ý (hồ sơ còn thiếu giới tính, vòng eo, huyết áp, tiền sử gia đình). Hai mô hình huấn luyện trên hai quần thể khác nhau (20 nghìn người có xét nghiệm và 3,17 triệu người tự khai) cho nguy cơ tương đối gần như trùng nhau (2,38 và 2,43 lần), nên hội đồng ghi thêm một bằng chứng (+) độc lập. Toàn bộ phần "suy nghĩ" của hội đồng mất khoảng 1,3 giây, phần lớn là nhánh Tri thức trong lần gọi đầu (nạp mô hình nhúng và duyệt đồ thị).

5.6. Kiểm thử

Bảng 5.3. Bộ kiểm thử tự động (43 kiểm thử, đều đạt)

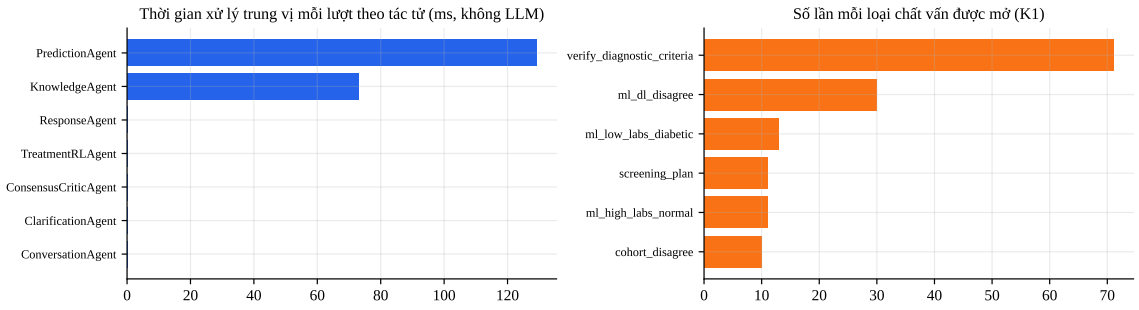
Tệp	Nội dung
`test_clinical_criteria.py`	Ngưỡng ADA cho FPG, HbA1c, OGTT; quy tắc "2 kết quả bất thường"
`test_extraction.py`	Trích chỉ số từ tin nhắn tiếng Việt, quy đổi đơn vị, đọc PDF, ảnh, DOCX, CSV
`test_scope.py`	Phát hiện hồ sơ ngoài phạm vi và kế hoạch câu hỏi làm rõ
`test_large_scale.py`	Hội đồng chỉ chất vấn khi mô hình NHANES và BRFSS nằm ở hai phía ngược nhau; phản hồi hạ độ tin cậy
`test_summary.py`	Câu tóm tắt cho từng loại thông điệp trong Phòng hội chẩn
`test_multi_agent.py`	Cả hội đồng chạy trên SQLite + Qdrant + Neo4j: hỏi lại khi thiếu dữ liệu, tranh biện 2 vòng, câu hỏi kiến thức không chạy nhánh dự đoán, guardrail ghi đè, câu hỏi tiếp nối trả lời từ bộ nhớ

5.7. Đánh giá đầu–cuối

Hệ thống được đánh giá trên **400 ca** lấy ngẫu nhiên từ tập kiểm tra NHANES (`PredictionAgent` chưa từng thấy), chỉ giữ người có cả FPG và HbA1c: 23,0% mắc ĐTĐ. Đánh giá chạy không LLM để kết quả tất định, trên đúng bộ lưu trữ SQLite + Qdrant + Neo4j của project.

Bảng 5.4. Đủ xét nghiệm (FPG + HbA1c): tính đúng đắn và vận hành của hội đồng (399 ca)

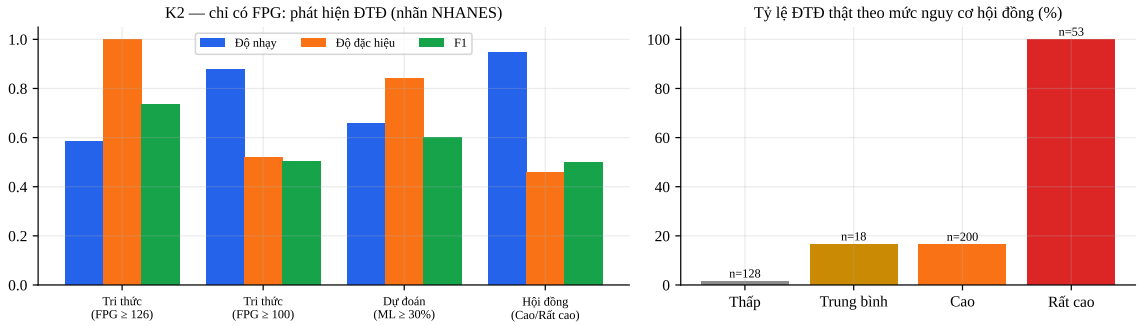
Chỉ số	Kết quả
Phân loại đường huyết đúng ngưỡng ADA	**399/399 (100%)**
Áp đúng quy tắc "2 xét nghiệm bất thường"	**71/71 (100%)**
Dữ liệu ngoài miền sinh lý	1 ca BMI 74,8 bị từ chối, hệ thống hỏi lại thay vì dự đoán
Số vòng tranh biện	0 vòng: 288 ca · 1 vòng: 100 · 2 vòng: 11
Chất vấn được mở	xác minh tiêu chí 71 · ML–DL bất đồng 30 · ML thấp nhưng xét nghiệm ĐTĐ 13 · ML cao nhưng xét nghiệm bình thường 11 → lịch tầm soát 11 · **NHANES–BRFSS bất đồng 10**
Mô hình quy mô lớn (BRFSS)	cùng phía với mô hình chính ở 97,5% ca; trung bình 4,6 điểm (+) mỗi ca (3,7 khi chưa có BRFSS)
Safety Guardrail	ưu tiên 77 ca · khám sớm 11 ca
Gợi ý can thiệp (RL sau ràng buộc)	lối sống 358 · metformin 5 · phối hợp tim–thận 25 · chuyên khoa 11
Thông điệp mỗi lượt	16–22 (trung bình 16,7); 146 cặp CHALLENGE/RESPONSE
Độ trễ một lượt (không LLM)	trung vị 136 ms, p95 192 ms; Prediction 129 ms (4 mô hình + giải thích) ‖ Knowledge 73 ms chạy song song



Hình 5.3. Thời gian xử lý của từng agent và tần suất các loại chất vấn

Bảng 5.5. Chỉ có FPG: giá trị của việc phối hợp nhiều agent (399 ca, 91 người ĐTĐ)

Cách ra quyết định	Độ nhạy	Đặc hiệu	PPV	F1	Số ca dương
Agent Tri thức đơn lẻ: FPG ≥ 126	0,582	**1,000**	**1,000**	**0,736**	53
Agent Tri thức đơn lẻ: FPG ≥ 100	0,879	0,520	0,351	0,502	228
Agent Dự đoán đơn lẻ: ML ≥ 30%	0,659	0,841	0,551	0,600	109
Hội đồng: mức Cao hoặc Rất cao	**0,945**	0,458	0,340	0,500	253



Hình 5.4. Độ nhạy, đặc hiệu của các cách ra quyết định, và tỉ lệ ĐTĐ thật theo mức nguy cơ của hội đồng

Nhận xét. Hai agent đơn lẻ bỏ sót những nhóm khác nhau. Ngưỡng FPG ≥ 126 chính xác tuyệt đối nhưng chỉ bắt được 58% người ĐTĐ: 38 người ĐTĐ có FPG dưới 126 (đang điều trị tốt, hoặc chỉ có HbA1c vượt ngưỡng). Hội đồng kết hợp xét nghiệm với mô hình đạt **độ nhạy 0,945**, bắt thêm **33/38** ca mà FPG bỏ sót. Điểm nguy cơ của hội đồng có AUC 0,876, cao hơn FPG đơn lẻ (0,862) và mô hình đơn lẻ (0,855). Cái giá là độ đặc hiệu 0,46. Đây là đánh đổi có chủ đích của một công cụ *sàng lọc*: mức Cao nghĩa là "nên làm thêm HbA1c hoặc OGTT", một xét nghiệm rẻ, trong khi bỏ sót người bệnh thì đắt hơn nhiều. Phân tầng có ý nghĩa rõ: mức Rất cao 100% là ĐTĐ thật (53/53), mức Thấp chỉ 1,6% (2/128).

Với tin nhắn tiếng Việt tự do sinh từ mẫu câu (399 tin), bộ trích xuất regex lấy đúng 100% các trường, và 91% tin nhắn cho cùng mức nguy cơ như khi nhập form. Phần còn lại khác mức vì tin nhắn ngẫu nhiên lược bớt thông tin (vòng eo, huyết áp, tiền sử), chứ không phải do trích sai.

5.8. Hạn chế và hướng phát triển

- **Mô hình**: trên NHANES, MLP chỉ nhỉnh hơn hồi quy logistic; cả hai bộ dữ liệu là của Hoa Kỳ, chưa hiệu chỉnh cho người Việt. BRFSS bổ sung 14.249 người gốc

Á ở tập kiểm tra (AUC 0,85) nhưng có nhãn tự khai, bỏ sót người mắc chưa được chẩn đoán, nên chỉ dùng làm ý kiến thứ hai.

- **RAG**: embedding MiniLM yếu với tiếng Việt chuyên ngành, làm tìm kiếm lai kém hơn toàn văn đơn lẻ trên bộ 60 câu. Hướng sửa: đổi sang E5 đa ngôn ngữ (hit@3 0,98 trong thí nghiệm của tiểu luận) hoặc gán trọng số cho RRF; mở rộng KB và bộ câu hỏi đánh giá.
- **RL**: môi trường và hàm thưởng tự đặt, chưa học từ dữ liệu điều trị thật. Hướng phát triển: mô phỏng đường huyết theo giờ để agent học kế hoạch ăn uống và vận động hằng ngày.
- **Trích xuất không LLM**: regex còn bỏ sót một số cách viết, ví dụ "Tôi nam 58 tuổi" chưa nhận ra giới tính trong ca demo. Khi có LLM, phần này do Gemini đảm nhận.
- **Đánh giá**: số liệu tin nhắn tự do là cận trên lạc quan, vì tin nhắn sinh từ mẫu câu mà regex được viết để hiểu.

KẾT LUẬN

Project đã hiện thực hóa sơ đồ hệ đa agent cho bệnh tiểu đường của đề bài thành một hệ thống chạy được đầu–cuối. Dữ liệu có sẵn của CDC, tổng cộng **3,19 triệu người** (NHANES 20.646 người có xét nghiệm và BRFSS 2011–2023 gồm 3.172.819 người), đi qua Deep Learning (`PredictionAgent`: GBDT + MLP, AUC 0,83 trên cả hai bộ) để dự đoán và phân loại nguy cơ. Mô hình quy mô lớn làm ý kiến thứ hai và đồng thuận với mô hình chính ở 97,5% ca. Tri thức từ hướng dẫn ADA 2024 và QĐ 5481/QĐ-BYT được tạo thành Knowledge Base gồm đồ thị 34 nút, 40 cạnh trên **Neo4j** và 36 đoạn văn được tìm kiếm lai trên **Qdrant** và **SQLite FTS5** (`KnowledgeAgent`, hit@3 0,90). Reinforcement learning kết hợp luật (`TreatmentRLAgent`) chọn bước can thiệp trong khuôn khổ an toàn. Các nhánh hội tụ ở một hội đồng tranh biện, nơi agent phản biện chất vấn agent chuyên môn khi chúng mâu thuẫn, rồi tư vấn và trợ giúp người dùng bằng câu trả lời có trích dẫn.

Đánh giá trên 400 ca NHANES chưa từng thấy cho thấy: lớp luật lâm sàng đúng tuyệt đối; phối hợp agent Tri thức với agent Dự đoán nâng độ nhạy phát hiện ĐTĐ từ 0,58 lên 0,95, với phân tầng có ý nghĩa (mức Rất cao 100% là ĐTĐ, mức Thấp 1,6%); tranh biện chỉ kích hoạt khi cần (26% ca); một lượt hội chẩn đầy đủ, chạy bốn mô hình dự đoán, mất khoảng 0,14 giây khi không dùng LLM. Giao diện Phòng hội chẩn giúp người xem thấy trực tiếp các agent trao đổi thông điệp, chất vấn và đi đến đồng thuận.

Bài học lớn nhất của project là trong bài toán y tế, giá trị của hệ đa agent không nằm ở việc có nhiều agent, mà ở chỗ các agent với dữ liệu và phương pháp khác nhau *kiểm tra chéo* lẫn nhau, mọi quyết định đều truy vết được tới một mô hình đã đánh giá hoặc một quy tắc có trích dẫn, và agent học luôn được đặt dưới sự giám sát của tri thức tường minh. Hướng phát triển tiếp theo gồm: thay embedding đa ngôn ngữ mạnh hơn cho RAG, hiệu chỉnh mô hình trên dữ liệu người Việt, và xây dựng môi trường mô phỏng đường huyết để agent RL học kế hoạch sinh hoạt hằng ngày.

TÀI LIỆU THAM KHẢO

[1] American Diabetes Association Professional Practice Committee, "2. diagnosis and classification of diabetes: Standards of care in diabetes—2024," *Diabetes Care*, tập 47, số Suppl. 1, S20–S42, 2024.

[2] Diabetes Prevention Program Research Group, "Reduction in the incidence of type 2 diabetes with lifestyle intervention or metformin," *New England Journal of Medicine*, tập 346, số 6, tr. 393–403, 2002.

[3] Bộ Y tế, *Quyết định số 5481/QĐ-BYT ban hành tài liệu chuyên môn "Hướng dẫn chẩn đoán và điều trị đái tháo đường típ 2"*, 2020.

[4] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ấn bản. Wiley, 2009.

[5] N. R. Jennings, K. Sycara, và M. Wooldridge, "A roadmap of agent research and development," *Autonomous Agents and Multi-Agent Systems*, tập 1, tr. 7–38, 1998.

[6] S. Russell và P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ấn bản. Pearson, 2021.

[7] B. Hayes-Roth, "A blackboard architecture for control," *Artificial Intelligence*, tập 26, số 3, tr. 251–321, 1985.

[8] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Transactions on Computers*, tập C-29, số 12, tr. 1104–1113, 1980.

[9] Y. Du và cộng sự, "Improving factuality and reasoning in language models through multiagent debate," trong *International Conference on Machine Learning (ICML)*, 2024.

[10] Foundation for Intelligent Physical Agents, *FIPA ACL message structure specification (SC00061G)*, 2002.

[11] S. Yao và cộng sự, "ReAct: Synergizing reasoning and acting in language models," trong *International Conference on Learning Representations (ICLR)*, 2023.

[12] Q. Wu và cộng sự, *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*, arXiv:2308.08155, 2023.

[13] S. Hong và cộng sự, "MetaGPT: Meta programming for a multi-agent collaborative framework," trong *International Conference on Learning Representations (ICLR)*, 2024.

[14] L. Wang và cộng sự, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, tập 18, tr. 186 345, 2024.

[15] T. Guo và cộng sự, "Large language model based multi-agents: A survey of progress and challenges," trong *International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[16] SQLite Consortium, *SQLite FTS5 extension*, `https://www.sqlite.org/fts5.html`, 2024.

[17] Qdrant Solutions GmbH, *Qdrant: Vector similarity search engine*, https://qdrant.tech/documentation/, 2025.

[18] Y. A. Malkov và D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, tập 42, số 4, tr. 824–836, 2020.

[19] Neo4j, Inc., *Neo4j graph database and the cypher query language*, https://neo4j.com/docs/, 2024.

[20] N. Reimers và I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," trong *EMNLP*, 2020.

[21] Cloudflare, *Cloudflare tunnel documentation*, https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/, 2025.

[22] Centers for Disease Control and Prevention, *National health and nutrition examination survey (NHANES) 2011–2018*, https://wwwn.cdc.gov/nchs/nhanes/, 2020.

[23] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, tập 29, số 5, tr. 1189–1232, 2001.

[24] G. Ke và cộng sự, "LightGBM: A highly efficient gradient boosting decision tree," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[25] A. Paszke và cộng sự, "PyTorch: An imperative style, high-performance deep learning library," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[26] I. Goodfellow, Y. Bengio, và A. Courville, *Deep Learning*. MIT Press, 2016.

[27] P. Lewis và cộng sự, "Retrieval-augmented generation for knowledge-intensive NLP tasks," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[28] Y. Gao và cộng sự, *Retrieval-augmented generation for large language models: A survey*, arXiv:2312.10997, 2023.

[29] S. Robertson và H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, tập 3, số 4, tr. 333–389, 2009.

[30] D. Edge và cộng sự, *From local to global: A graph RAG approach to query-focused summarization*, arXiv:2404.16130, 2024.

[31] G. V. Cormack, C. L. A. Clarke, và S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," trong *ACM SIGIR*, 2009, tr. 758–759.

[32] C. J. C. H. Watkins và P. Dayan, "Q-learning," *Machine Learning*, tập 8, tr. 279–292, 1992.

[33] R. S. Sutton và A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ấn bản. MIT Press, 2018.